Noakhali Science & Technology University
Department of Computer Science & Telecommunication Engineering

Assignment On:

Convolutional Neural Networks (CNNs)

Course Code: CSTE 4203

Submitted By

Mohammad Borhan Uddin
ID: ASH2101008M
Session: 2020-21

Submitted To

Dr. Fateha Khanam Bappee
Associate Professor
Department of Computer Science & Telecommunication Engineering

Date : 29th July, 2026

Contents

1 Introduction

1.1 Definition

A Convolutional Neural Network (CNN) is a type of deep learning model specifically designed to process and analyze structured grid-like data, particularly images. Unlike traditional neural networks, CNNs can automatically learn and extract important features such as edges, textures, shapes, and objects directly from raw input data. This automatic feature extraction capability makes CNNs highly effective for computer vision tasks, including image classification, object detection, facial recognition, and medical image analysis.

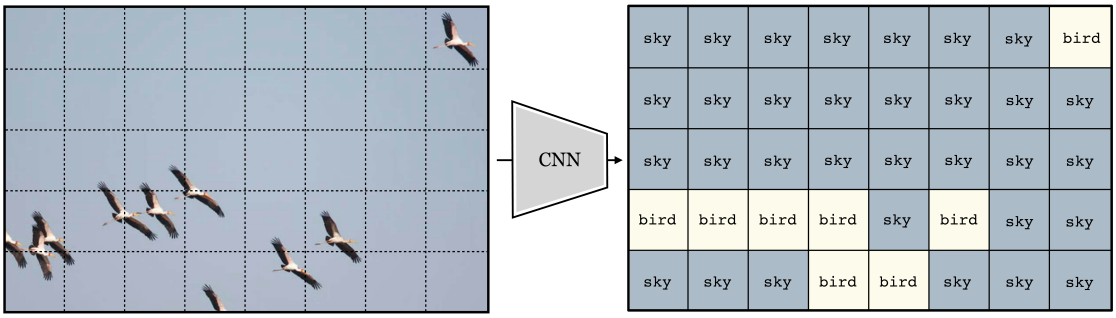

Figure 1: Input-Output Mapping of a CNN, Source: https://visionbook.mit.edu/

1.2 Why is it called a Convolutional Neural Network?

The term Convolutional Neural Network originates from the convolution operation, which is the core mathematical process used in the network. During convolution, small matrices called filters or kernels slide across the input image to detect local patterns and features. Each filter specializes in identifying specific characteristics, such as edges, corners, or textures, and produces a feature map that highlights the presence of those features. Since convolution is the primary operation responsible for feature extraction, the network is referred to as a Convolutional Neural Network (CNN).

1.3 Difference Between CNN and Traditional Artificial Neural Networks (ANN)

Although both CNNs and Artificial Neural Networks (ANNs) are types of deep learning models, they differ significantly in their architecture and applications. Traditional ANNs consist of fully connected layers in which every neuron is connected to every neuron in the next layer. As a result, ANNs require input data to

be converted into a one-dimensional vector, leading to a large number of parameters and the loss of important spatial information. Consequently, ANNs are more suitable for structured or tabular data.

In contrast, CNNs are specifically designed for image and spatial data. They use convolutional layers with shared weights and local connections to preserve the spatial relationships between pixels while significantly reducing the number of learnable parameters. CNNs automatically extract hierarchical features from images, eliminating the need for manual feature engineering. Therefore, CNNs generally achieve higher accuracy and better computational efficiency than traditional ANNs in image-related tasks.

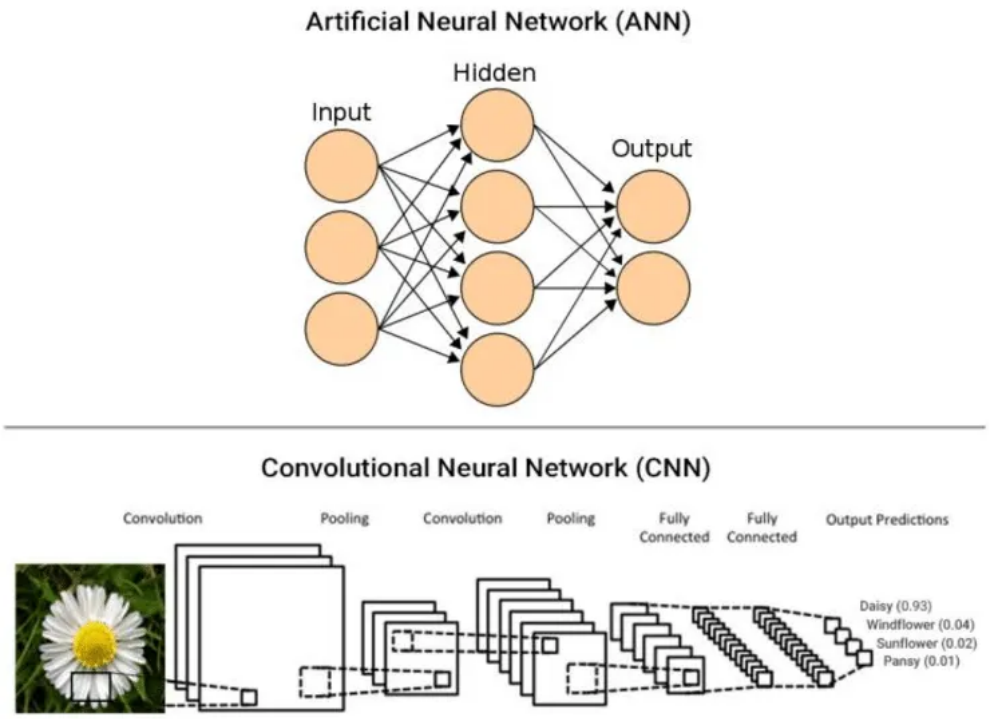

Figure 2: Architectural differences between CNN and ANN, Source : Divyanshu Bhardwaj on *Medium : Why CNN performs better than ANN on image classification*

2 Architecture & Layers

2.1 CNN Architecture

The architecture of a Convolutional Neural Network (CNN) consists of a sequence of specialized layers that work together to extract features from input data and perform classification or prediction tasks.

The Input Layer receives the raw image as a matrix of pixel values. The Convolutional Layers apply filters (kernels) to detect important local features such

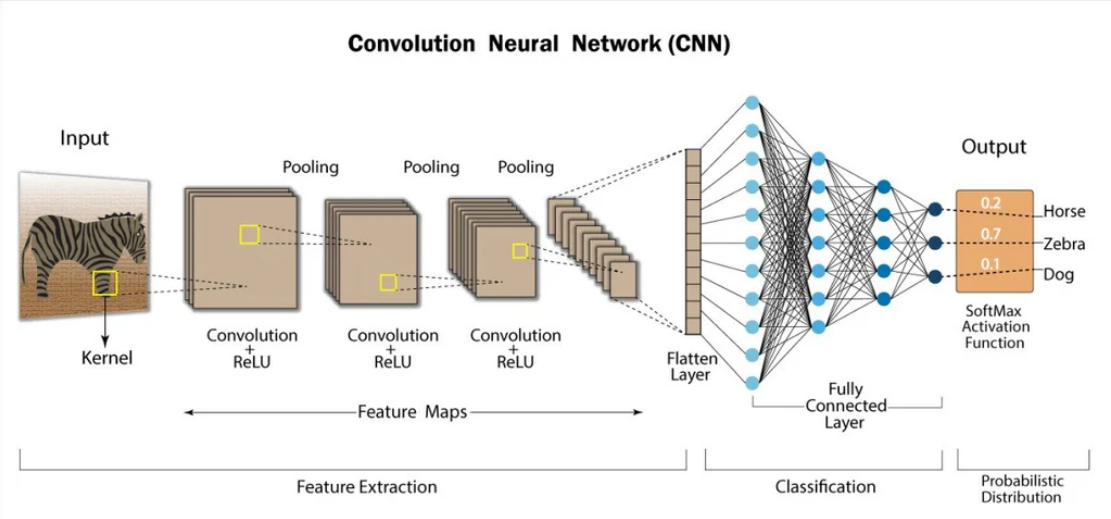

Figure 3: CNN Architecture

as edges, textures, and shapes. After each convolution, an Activation Function, typically the Rectified Linear Unit (ReLU), introduces non-linearity, enabling the network to learn complex patterns.

Next, the Pooling Layers reduce the spatial dimensions of the feature maps while preserving the most significant information. This process decreases computational complexity and helps reduce overfitting.

After several convolution and pooling operations, the extracted feature maps are converted into a one-dimensional vector by the Flatten Layer. This vector is then passed to one or more Fully Connected Layers, which combine the extracted features to perform classification or regression. Finally, the Output Layer produces the network's prediction, often using activation functions such as Softmax for multi-class classification or Sigmoid for binary classification.

Overall, the hierarchical architecture of CNNs enables the network to automatically learn increasingly complex features, making them highly effective for image recognition and other computer vision tasks.

Workflow of Each Layer

2.2 Input Layer

The Input Layer is the first layer of a CNN and serves as the entry point for the network. It receives the raw input image as a matrix of pixel values. For grayscale images, the input consists of a two-dimensional matrix, whereas color images are represented as three-dimensional matrices corresponding to the red, green, and blue (RGB) channels. The input layer itself performs no computation; instead, it passes the image data to the subsequent convolutional layer while preserving the

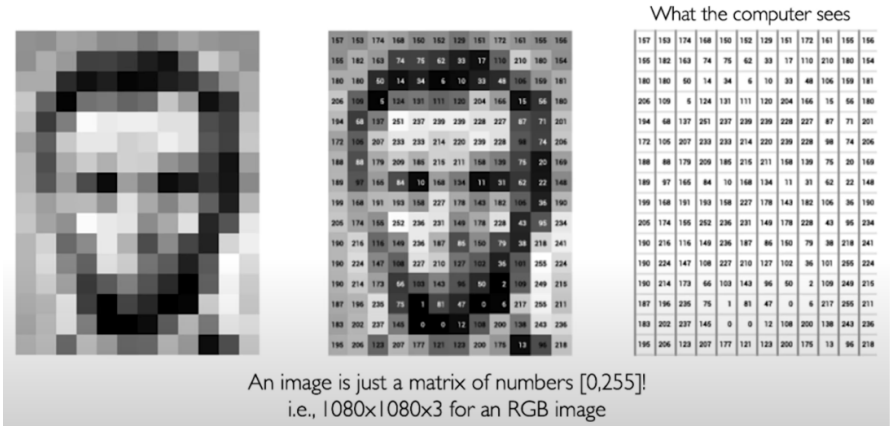

Figure 4: Input Imagem, Source: https://authurwhywait.github.io/

original spatial information.

2.3 Convolutional Layer

The **Convolutional Layer** is the fundamental building block of a Convolutional Neural Network (CNN). It is responsible for automatically extracting meaningful features from the input image, such as edges, corners, textures, patterns, and object parts. Unlike traditional Artificial Neural Networks (ANNs), which connect every neuron to every neuron in the next layer, the convolutional layer employs the principles of *local connectivity* and *weight sharing*. These properties significantly reduce the number of trainable parameters while preserving the spatial relationships among neighboring pixels.

The primary operation performed in this layer is the **convolution operation**. During convolution, a small matrix called a *kernel* or *filter* slides across the input image. At every spatial location, the kernel performs element-wise multiplication with the corresponding region of the input image, and the products are summed to generate a single output value. Repeating this process over the entire image produces a two-dimensional feature map that represents the response of the filter to different regions of the input.

The convolution operation enables the CNN to automatically learn feature detectors during training. In the initial layers, filters generally learn simple features such as horizontal and vertical edges, whereas deeper layers learn more abstract and complex features such as object parts and complete objects.

2.3.1 Mathematical Representation of Convolution

Consider an input image

$$I \in R^{H \times W}$$

where

- H = height of the image,
- W = width of the image.

Let the convolution kernel be

$$K \in R^{k \times k}$$

where k denotes the kernel size.

The convolution operation is mathematically defined as

$$S(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m, j+n) \times K(m,n) \tag{1}$$

where

- $I(i+m, j+n)$ represents the pixel value of the input image,
- $K(m,n)$ denotes the kernel weight,
- $S(i,j)$ is the output feature map.

The resulting feature map contains the learned features extracted from the original image.

2.3.2 Example of Convolution

Suppose the input image is

$$I = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and the kernel is

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

The convolution at the first position is

$$(1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times (-1))$$

$$= 1 + 0 + 0 - 5 = -4$$

The kernel continues sliding across the image until all possible locations are processed, producing the complete feature map.

2.3.3 Kernel (Filter)

A kernel, also known as a filter, is a small matrix containing trainable weights. During training, these weights are updated using backpropagation so that the kernel learns to detect specific image features.

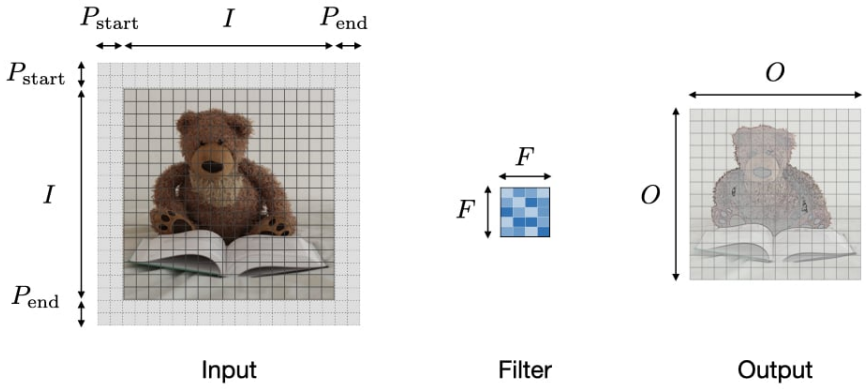

Figure 5: Filter/Kernel, from stanford.edu

Common kernel sizes include

- 3×3
- 5×5
- 7×7

Small kernels are preferred in modern CNN architectures because they require fewer parameters while achieving high feature extraction performance.

2.3.4 Stride

Stride specifies the number of pixels by which the kernel moves during convolution.

If

$$S = 1$$

the kernel moves one pixel at a time.

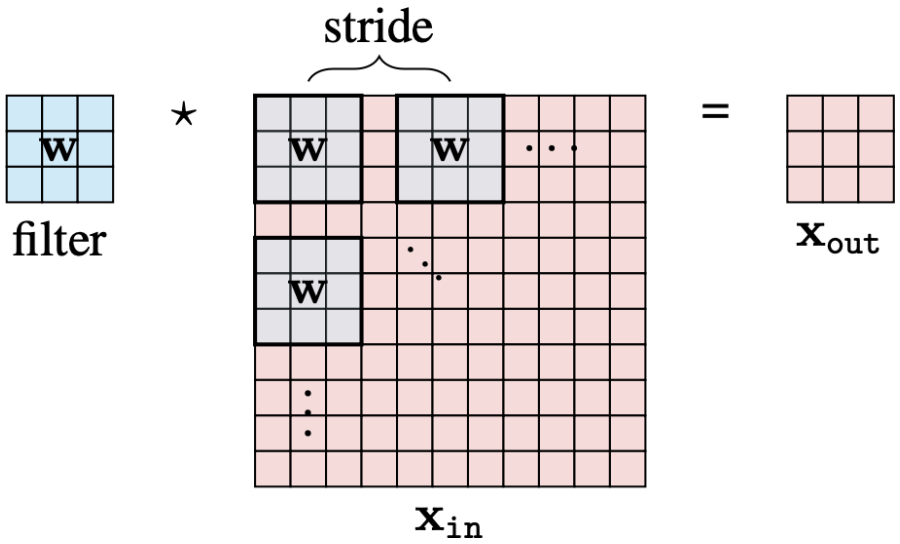

Figure 6: Strided Convolution, Source: https://visionbook.mit.edu/

If

$$S = 2$$

the kernel skips one pixel after each movement, thereby reducing the output dimensions.

A larger stride decreases computational cost but may also reduce the amount of extracted information.

2.3.5 Padding

Padding refers to adding extra pixels around the border of the input image before performing convolution.

Two common padding methods are

1. Valid Padding
2. Same Padding

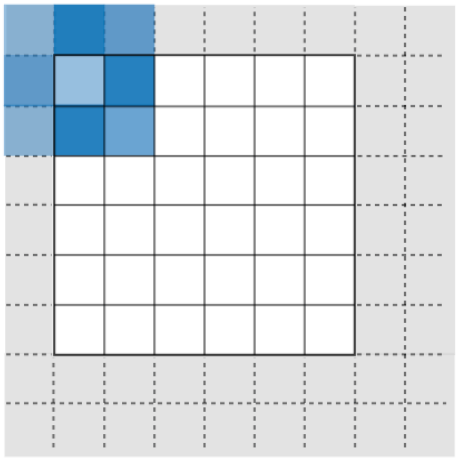

Figure 7: Padding, Source: stanford.edu

Valid Padding No additional pixels are added.

$$P = 0$$

The output feature map becomes smaller than the input image.

Same Padding Zeros are added around the image boundary.

This ensures that the spatial dimensions of the output feature map remain approximately equal to those of the input image.

2.3.6 Output Dimension

The spatial dimensions of the output feature map are computed using

$$O = \left\lfloor \frac{N - K + 2P}{S} \right\rfloor + 1 \tag{2}$$

where

- N = input size
- K = kernel size
- P = padding
- S = stride

Example Suppose

$$N = 32, \qquad K = 3, \qquad P = 1, \qquad S = 1$$

Then

$$O = \left\lfloor \frac{32 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$= 31 + 1 = 32$$

Thus, the output feature map has dimensions

$$32 \times 32.$$

2.3.7 Multiple Filters

A CNN generally employs many filters simultaneously.

If the network contains

$$F = 64$$

filters, then

$$64$$

different feature maps are produced.

Each filter learns a different characteristic of the input image.

2.3.8 Weight Sharing

Unlike a fully connected neural network, CNNs use the same kernel weights across all image locations.

If a kernel contains

$$3 \times 3$$

weights,

only

parameters are learned regardless of the image size.

Weight sharing greatly reduces memory consumption and improves computational efficiency.

2.3.9 Local Receptive Field

Each neuron in the convolutional layer processes only a small local region of the input image rather than the entire image.

This local connectivity enables CNNs to effectively learn spatial patterns while reducing computational complexity.

2.3.10 Advantages of the Convolution Layer

The convolutional layer offers several important advantages:

- Automatically extracts useful image features.
- Preserves spatial information.
- Reduces the number of trainable parameters through weight sharing.
- Learns hierarchical feature representations.
- Improves computational efficiency compared with fully connected networks.
- Provides translation invariance when combined with pooling layers.

2.4 Activation Layer (ReLU)

After the convolution operation, the extracted feature maps are passed through an **Activation Layer**. The purpose of this layer is to introduce *non-linearity* into the network. Without an activation function, a Convolutional Neural Network (CNN) would behave as a linear model regardless of the number of convolutional layers, making it incapable of learning complex patterns present in real-world data.

Among the various activation functions, the **Rectified Linear Unit (ReLU)** is the most widely used due to its computational simplicity, efficient gradient propagation, and superior performance in deep neural networks. The ReLU activation function transforms all negative input values to zero while leaving positive values unchanged.

The ReLU layer is typically applied immediately after each convolutional layer. As a result, every feature map generated by convolution is transformed before being forwarded to the pooling layer or the next convolutional layer.

2.4.1 Mathematical Representation of ReLU

The Rectified Linear Unit is mathematically defined as

$$f(x) = \max(0, x) \tag{3}$$

where

- x is the input to the activation function,
- $f(x)$ is the activated output.

The function operates as follows:

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Thus,

- all negative values become zero,
- zero remains zero,
- positive values remain unchanged.

2.4.2 Example of ReLU

Consider the following feature map produced by a convolutional layer:

$$X = \begin{bmatrix} 2 & -3 & 5 \\ -1 & 4 & -2 \\ 7 & -6 & 1 \end{bmatrix}$$

Applying the ReLU activation function gives

$$ReLU(X) = \begin{bmatrix} 2 & 0 & 5 \\ 0 & 4 & 0 \\ 7 & 0 & 1 \end{bmatrix}$$

All negative values are replaced with zero, while positive values remain unchanged.

2.4.3 Derivative of ReLU

During backpropagation, the derivative of the activation function is required to compute gradients.

The derivative of ReLU is

$$f'(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases} \tag{4}$$

The derivative at

$$x = 0$$

is mathematically undefined. However, in practice, most deep learning frameworks assign it either 0 or 1, with zero being the most common choice.

2.4.4 Role of ReLU in CNNs

The ReLU activation function performs several important tasks within a CNN.

First, it introduces non-linearity into the network, allowing CNNs to model complex relationships between the input and output. Real-world images contain highly non-linear patterns that cannot be represented using only linear transformations.

Second, ReLU suppresses weak or irrelevant activations by setting negative values to zero. This enables the network to focus on the most informative features extracted by the convolutional layers.

Third, ReLU improves computational efficiency because its implementation requires only a simple comparison operation instead of expensive exponential or trigonometric calculations.

2.4.5 Variants of ReLU

To overcome the limitations of the standard ReLU, several improved activation functions have been proposed.

Leaky ReLU Instead of assigning zero to negative inputs, Leaky ReLU allows a small negative slope.

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \tag{5}$$

where

$$\alpha \approx 0.01$$

This prevents neurons from becoming permanently inactive.

Parametric ReLU (PReLU) PReLU is similar to Leaky ReLU, but the parameter

$$\alpha$$

is learned automatically during training.

$$f(x) = \begin{cases} x, & x > 0 \\ ax, & x \leq 0 \end{cases} \tag{6}$$

where

$$a$$

is a trainable parameter.

Exponential Linear Unit (ELU) The ELU activation function is defined as

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases} \tag{7}$$

ELU produces smoother gradients for negative inputs and often improves convergence in very deep networks.

2.4.6 Advantages of ReLU

The ReLU activation function provides several important benefits:

- Introduces non-linearity into the network.
- Accelerates the convergence of deep neural networks.
- Requires very low computational cost.
- Helps alleviate the vanishing gradient problem.
- Produces sparse activation, improving computational efficiency.
- Performs well in most modern CNN architectures.

2.4.7 Limitations of ReLU

Despite its effectiveness, ReLU has several limitations.

- Neurons may become permanently inactive (Dying ReLU).
- Negative information is completely discarded.
- Performance may degrade if a large number of neurons become inactive.

2.5 Pooling Layer

The **Pooling Layer** is a downsampling layer in a Convolutional Neural Network (CNN) that reduces the spatial dimensions (height and width) of feature maps while preserving the most important information. It is usually placed after the activation layer (ReLU) and helps decrease computational complexity, reduce memory usage, and minimize overfitting.

Unlike the convolutional layer, the pooling layer does not contain trainable parameters. Instead, it applies a fixed operation over small regions of the feature map to generate a smaller representation. By reducing the size of feature maps, pooling enables the network to process data more efficiently and improves its ability to recognize features regardless of small translations or distortions in the input image.

2.5.1 Pooling Operation

A pooling window (or filter) slides across the feature map with a specified stride. For each window, a single representative value is selected according to the pooling method. The output is a reduced feature map that retains the most significant information.

2.5.2 Types of Pooling

The two most commonly used pooling methods are:

Max Pooling Max Pooling selects the maximum value from each pooling window. It preserves the strongest activation and is the most widely used pooling technique in modern CNN architectures.

Mathematically,

$$P(i,j) = \max_{(m,n)\in R} X(m,n) \tag{8}$$

where

- X is the input feature map,
- R represents the pooling region,
- $P(i,j)$ is the pooled output.

Example For a 2×2 pooling window,

$$\begin{bmatrix} 2 & 5 \\ 7 & 1 \end{bmatrix}$$

the output is

7.

Average Pooling Average Pooling computes the average value of all elements within the pooling window.

It is defined as

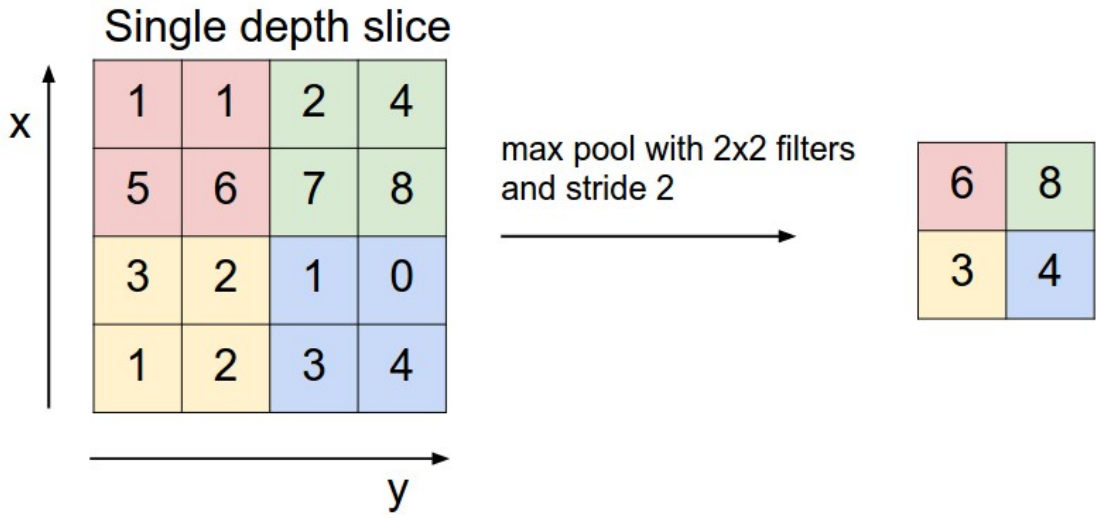

Figure 8: Max-Pooling, Source : https://cs231n.github.io/convolutional-networks/

$$P(i,j) = \frac{1}{|R|} \sum_{(m,n)\in R} X(m,n) \tag{9}$$

where $|R|$ denotes the number of elements in the pooling region.

Although Average Pooling preserves more overall information, Max Pooling generally provides better performance in image classification tasks.

2.5.3 Stride in Pooling

Similar to convolution, pooling uses a stride to determine how far the pooling window moves after each operation.

For example,

- Stride = 1 produces overlapping pooling regions.
- Stride = 2 is commonly used and reduces the spatial dimensions by approximately half.

2.5.4 Output Dimension

The output size of a pooling layer is calculated using

$$O = \left\lfloor \frac{N-F}{S} \right\rfloor + 1 \tag{10}$$

where

- N = input size,
- F = pooling window size,
- S = stride.

2.5.5 Advantages of Pooling

The pooling layer offers several important benefits:

- Reduces the spatial dimensions of feature maps.
- Decreases computational cost and memory usage.
- Helps reduce overfitting.
- Improves robustness to small translations and distortions in the input image.
- Preserves the most informative features for subsequent layers.

2.6 Flatten Layer

The **Flatten Layer** serves as a bridge between the feature extraction stage and the classification stage of a Convolutional Neural Network (CNN). After passing through several convolutional, activation, and pooling layers, the data is represented as a multi-dimensional feature map. However, the fully connected layer requires a one-dimensional input vector. Therefore, the Flatten Layer converts the multi-dimensional feature map into a one-dimensional vector without altering the extracted feature values.

Unlike convolutional and fully connected layers, the Flatten Layer does not perform any mathematical computation or learn any trainable parameters. Instead, it simply rearranges the elements of the feature map into a single continuous vector.

2.6.1 Flattening Operation

Suppose the output of the final pooling layer is a feature map of dimensions

$$H \times W \times C,$$

where

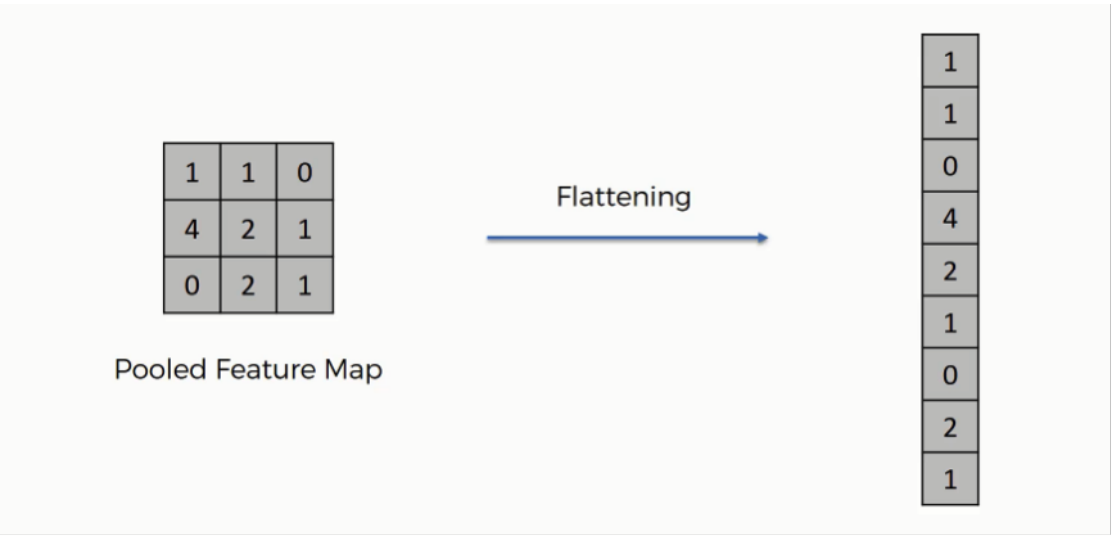

Figure 9: Flatten Layer, Source: https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-3-flattening

- H = height of the feature map,
- W = width of the feature map,
- C = number of channels (feature maps).

The Flatten Layer transforms this three-dimensional tensor into a one-dimensional vector of length

$$L = H \times W \times C \tag{11}$$

where L is the total number of elements in the flattened vector.

2.6.2 Example

Assume the output feature map has dimensions

$$4 \times 4 \times 16.$$

The Flatten Layer converts it into a vector of length

$$4 \times 4 \times 16 = 256.$$

Thus,

$$4 \times 4 \times 16 \longrightarrow 256.$$

This vector is then used as the input to the fully connected layer.

2.6.3 Advantages of Flattening

The Flatten Layer provides several benefits:

- Converts multi-dimensional feature maps into a one-dimensional vector.
- Enables the transition from feature extraction to classification.
- Introduces no additional trainable parameters.
- Preserves all extracted feature values during the transformation.

2.7 Fully Connected Layer

The **Fully Connected (FC) Layer**, also known as the *Dense Layer*, is the final learning component of a Convolutional Neural Network (CNN). After the Flatten Layer converts the extracted feature maps into a one-dimensional vector, the Fully Connected Layer combines these features to perform classification or prediction.

In this layer, every neuron is connected to every neuron in the previous layer. Each connection has an associated weight and bias, which are learned during training through backpropagation. Using these learned parameters, the network determines the relationship between the extracted features and the target classes.

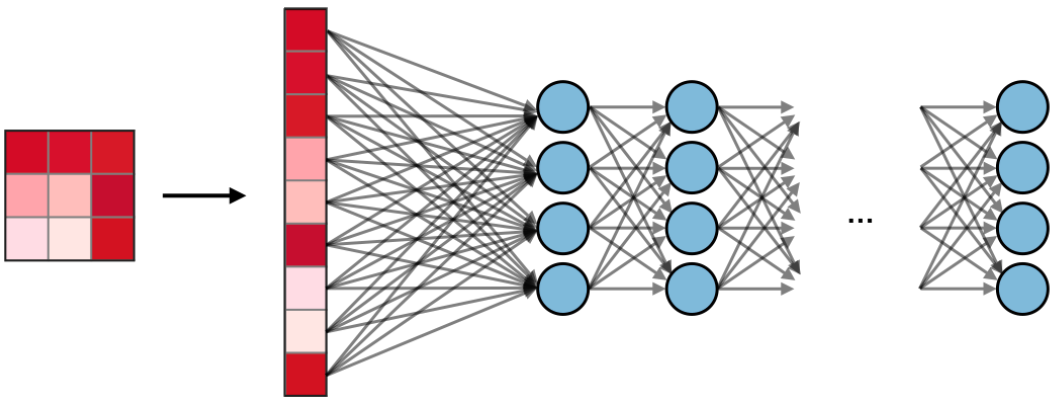

Figure 10: Fully Connected Layer. Source: Stanford University

2.7.1 Mathematical Representation

Let the input feature vector be

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]^T.$$

The output of a neuron in the Fully Connected Layer is computed as

$$z = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{12}$$

where

- $\mathbf{W}$ is the weight matrix,
- $\mathbf{x}$ is the input feature vector,
- $\mathbf{b}$ is the bias vector,
- z is the linear output.

An activation function is then applied to produce the final output:

$$a = f(z) \tag{13}$$

where $f(\cdot)$ is commonly ReLU for hidden layers or Softmax/Sigmoid in the output layer.

2.7.2 Learning Process

During training, the Fully Connected Layer adjusts its weights and biases using the backpropagation algorithm. The objective is to minimize the loss function by updating the parameters through gradient descent, allowing the network to improve its prediction accuracy over time.

2.7.3 Advantages

The Fully Connected Layer provides several important benefits:

- Combines all extracted features for final decision-making.
- Learns complex relationships between features and output classes.
- Produces class scores used for prediction.

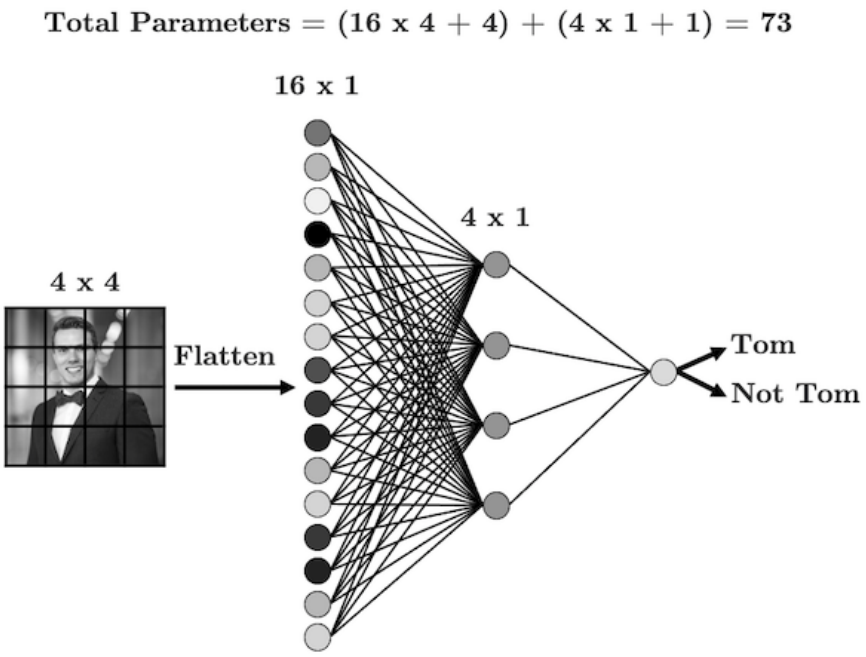

Figure 11: Output, Source : Chapter 5: Introduction to Convolutional Neural Networks, Deep Learning With PyTorch

2.8 Output Layer

The **Output Layer** is the final layer of a Convolutional Neural Network (CNN). Its primary function is to produce the network's final prediction based on the features learned by the previous layers. The number of neurons in the output layer depends on the number of target classes in the classification problem.

For binary classification, the output layer typically consists of a single neuron with a **Sigmoid** activation function. For multi-class classification, it contains one neuron for each class and commonly uses the **Softmax** activation function to generate class probabilities.

2.8.1 Sigmoid Activation

For binary classification, the Sigmoid function maps the output to a probability between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{14}$$

where

- z is the input to the output neuron,
- $\sigma(z)$ is the predicted probability.

If the predicted probability exceeds a predefined threshold (commonly 0.5), the sample is assigned to the positive class; otherwise, it is assigned to the negative class.

2.8.2 Softmax Activation

For multi-class classification, the Softmax function converts the output scores into a probability distribution.

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \tag{15}$$

where

- z_i is the score of the i^{th} class,
- C is the total number of classes,
- $P(y_i)$ is the predicted probability of class i.

The probabilities of all classes sum to one, and the class with the highest probability is selected as the final prediction.

2.8.3 Prediction Process

During inference, the output layer computes the probability of each possible class. The predicted class is determined by selecting the class with the highest probability:

$$\hat{y} = \arg\max_{i} P(y_i) \tag{16}$$

where $\hat{y}$ denotes the predicted class label.

2.8.4 Advantages

The Output Layer provides several important benefits:

- Produces the final prediction of the CNN.
- Converts network outputs into interpretable probabilities.
- Supports both binary and multi-class classification tasks.

3 Applications of Convolutional Neural Networks

Convolutional Neural Networks (CNNs) have become one of the most influential deep learning architectures due to their exceptional ability to automatically learn hierarchical features from raw data. Their capability to extract both low-level and high-level representations makes them suitable for a wide range of computer vision tasks. CNNs have significantly improved the accuracy and efficiency of image analysis systems and are widely employed in healthcare, autonomous vehicles, surveillance, robotics, agriculture, and many other fields. This section discusses several major applications of CNNs, including image classification, video classification, object detection, semantic segmentation, and other emerging applications.

3.1 Image Classification

Image classification is one of the most fundamental applications of Convolutional Neural Networks. The objective of image classification is to assign an input image to one predefined category or class. Given an input image, the CNN automatically extracts important visual features through multiple convolutional and pooling layers before predicting the most probable class.

During classification, the convolutional layers learn low-level features such as edges and textures in the early stages, while deeper layers learn more complex features including shapes, object parts, and complete objects. These extracted features are then passed through fully connected layers, and the final prediction is generated using the Softmax activation function.

For a multi-class classification problem, the probability of class i is computed using the Softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \tag{17}$$

where

- z_i is the output score for class i,
- C is the total number of classes,
- $P(y_i)$ represents the predicted probability of class i.

The image is assigned to the class with the highest predicted probability.

Image classification is widely used in numerous real-world applications, including:

- Medical image diagnosis
- Face recognition systems
- Handwritten digit recognition
- Plant disease identification
- Wildlife species classification
- Industrial quality inspection

Popular benchmark datasets for image classification include MNIST, CIFAR-10, CIFAR-100, and ImageNet.

3.2 Object Detection

Object detection extends image classification by not only identifying the objects present in an image but also determining their precise locations. Instead of assigning a single label to an entire image, object detection predicts multiple objects together with their corresponding bounding boxes.

A CNN first extracts visual features from the image, after which specialized detection networks estimate both the object category and the coordinates of each bounding box. Therefore, object detection simultaneously solves two problems:

Figure 12: Object Detection, Source : authurwhywait.github.io

- Object classification
- Object localization

The performance of object detection models is commonly evaluated using the **Intersection over Union (IoU)** metric, defined as

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{18}$$

A larger IoU value indicates better agreement between the predicted bounding box and the ground-truth bounding box.

Several CNN-based object detection algorithms have been developed over the years, including:

- R-CNN
- Fast R-CNN
- Faster R-CNN
- YOLO (You Only Look Once)
- SSD (Single Shot MultiBox Detector)

3.3 Semantic Segmentation

Semantic segmentation is a computer vision task in which every pixel of an image is assigned to a specific class. Unlike image classification, which predicts a single label, or object detection, which predicts bounding boxes, semantic segmentation performs dense pixel-level classification.

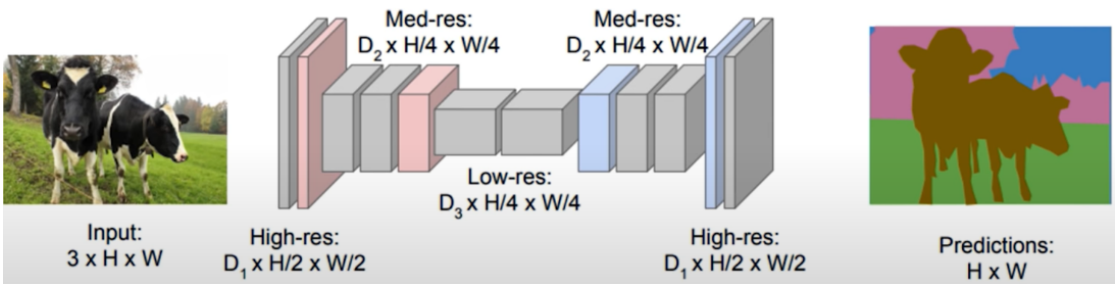

Figure 13: Semantic Segmentation, Source : authurwhywait.github.io

In semantic segmentation, the CNN generates a feature representation of the image and then reconstructs a segmentation map in which each pixel corresponds to a semantic category. Consequently, objects are identified with much greater precision than in object detection.

Popular CNN-based semantic segmentation models include:

- Fully Convolutional Network (FCN)
- U-Net
- DeepLab
- Mask R-CNN

Due to its pixel-level accuracy, semantic segmentation plays a crucial role in applications requiring detailed image understanding.

3.4 Other Applications of CNNs

Some additional applications include:

- Optical Character Recognition (OCR)
- Facial emotion recognition
- Medical image analysis
- Biometric authentication
- Fingerprint and iris recognition
- Remote sensing and satellite image classification
- Industrial defect detection
- Crop disease monitoring
- Robotics and autonomous navigation
- Augmented and virtual reality

4 Recent Advancements and State-of-the-Art CNN Models

Since the introduction of early CNN architectures, significant advancements have been made to improve accuracy, computational efficiency, and scalability. Modern CNN models have introduced innovative architectural designs such as deeper networks, residual connections, dense feature reuse, and efficient scaling strategies. These improvements have enabled CNNs to achieve state-of-the-art performance on various computer vision tasks, including image classification, object detection, and semantic segmentation.

4.1 Evolution of CNN Architectures

The development of CNNs has progressed through several milestone architectures:

- **LeNet-5 (1998):** One of the earliest CNNs, designed for handwritten digit recognition.

- **AlexNet (2012):** Demonstrated the effectiveness of deep CNNs by winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012.
- **VGGNet (2014):** Introduced deeper networks using small 3×3 convolution filters.
- **GoogLeNet (2014):** Proposed the Inception module to improve computational efficiency.
- **ResNet (2015):** Introduced residual (skip) connections, allowing very deep neural networks to be trained successfully.
- **DenseNet (2017):** Connected each layer to every subsequent layer, improving feature reuse and gradient flow.
- **EfficientNet (2019):** Achieved excellent performance through compound scaling of network depth, width, and input resolution.
- **ConvNeXt (2022):** Modernized CNN architectures by incorporating design ideas inspired by Vision Transformers while retaining convolutional operations.

4.2 Major State-of-the-Art CNN Models

AlexNet (2012) AlexNet popularized deep CNNs by introducing ReLU activation, dropout regularization, and GPU-based training. It significantly outperformed previous methods on the ImageNet dataset and demonstrated the potential of deep learning for computer vision.

VGGNet (2014) VGGNet employed a simple architecture consisting of multiple stacked 3×3 convolutional layers. Although computationally expensive, it achieved high classification accuracy and became a widely used feature extraction model.

GoogLeNet (2014) GoogLeNet introduced the Inception module, which performs multiple convolution operations in parallel using different kernel sizes. This design greatly reduced the number of parameters while maintaining high performance.

ResNet (2015) ResNet introduced residual learning through skip connections, allowing gradients to flow directly across layers. The residual mapping is expressed as

$$H(x) = F(x) + x \tag{19}$$

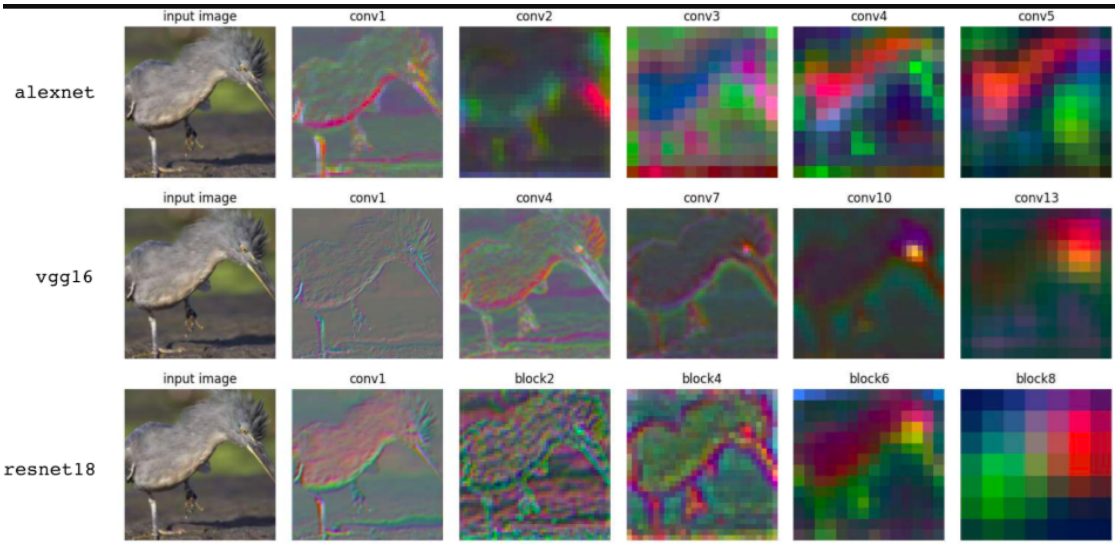

Figure 14: CNN Models, Source : https://visionbook.mit.edu/

where $F(x)$ represents the learned residual function and x is the input. This innovation enabled the successful training of networks exceeding 100 layers.

DenseNet (2017) DenseNet improves feature reuse by connecting each layer with all previous layers. This architecture enhances information flow, reduces the number of parameters, and alleviates the vanishing gradient problem.

EfficientNet (2019) EfficientNet introduced compound scaling, which balances network depth, width, and input resolution to achieve higher accuracy with fewer computational resources.

ConvNeXt (2022) ConvNeXt modernized traditional CNN architectures by incorporating design principles inspired by Vision Transformers. It demonstrated that carefully designed CNNs remain highly competitive for modern computer vision tasks.

References

[1] Afshine Amidi and Shervine Amidi, *CS 230 Cheatsheet: Convolutional Neural Networks*. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/

[2] Antonio Torralba, Phillip Isola, and William T. Freeman, *Computer Vision: Convolutional Neural Networks*. Available: https://visionbook.mit.edu/convolutional_neural_nets.html

[3] Andrej Karpathy, *CS231n: Convolutional Neural Networks for Visual Recognition*. Available: https://cs231n.github.io/convolutional-networks/

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/contents/convnets.html

[5] Arthur Whywait, *Introduction to Deep Learning*. Available: https://authurwhywait.github.io/blog/2021/12/05/introduction_dl/

[6] Thomas Beuzen, *Deep Learning with PyTorch: Chapter 5 – Convolutional Neural Networks*. Available: https://www.tomasbeuzen.com/deep-learning-with-pytorch/chapters/chapter5_cnns-pt1.html

[7] Polo Club of Data Science, *CNN Explainer*. Available: https://poloclub.github.io/cnn-explainer/

[8] Divyanshu Bansal, *Why CNN Performs Better than ANN on Image Classification*. Available: https://medium.com/@divyanshub2311/why-cnn-performs-better-than-ann-on-image-classification-7f92e5a92904